

Ordering Sensitivity in Subgraph-Isomorphism Layout Search:

A Characterization of Catastrophic Failure Regions in Quantum Circuit Transpilation

TN Holdings

love.os.architect@proton.me

<https://github.com/TN-Holdings-LLC/psf-zero>

2026-09-21

Abstract

Qubit layout selection in quantum circuit transpilation is commonly formulated as subgraph isomorphism and solved with VF2-family search. We report that Qiskit's `VF2Layout` pass exhibits a sharp, reproducible failure region: on interaction graphs that exactly saturate a device's maximum matching, median search time rises from 34.1 ms to 9236 ms—a factor of 271—and terminates by declaring no solution exists, on instances where a valid embedding demonstrably does exist. The failure is not a property of subgraph-isomorphism search as such. Reading Qiskit's current Rust source, we find `VF2Layout` does not call the public `rustworkx` mapper but a separate implementation with a hardcoded VF2++ node-ordering strategy, applied unconditionally. On the same 26 saturated instances, a single call to `rustworkx.vf2_mapping` with a fixed, unshuffled node order (`id_order=True`) finds a valid embedding in all 26 cases, at a median of 0.0292 ms—roughly five orders of magnitude faster than the failing search on the 19 instances where Qiskit reports no solution. We show by faithful simulation of Qiskit's own `Vf2ppSorter` that VF2++ ordering defers the highly symmetric, degree-one region of these interaction graphs to the end of its traversal, and we report that this mechanism alone does not account for the observed instance-level divergence: graph-isomorphic instance pairs differing only in which physical qubits host which component produce opposite outcomes, reproducibly. A permutation-based analysis across all 325 instance pairs finds that no graph-level feature we computed explains more than 4.5% of outcome variance, with none surviving correction for multiple comparisons. We report a cross-SDK comparison in which Cirq's placement search exceeds a 10-second budget across a substantially wider occupancy range than Qiskit's failure region, and topology results on brick and heavy-hex lattices. All predictions were pre-registered before measurement; all raw data, pre-registrations, and analysis scripts are public.

Introduction

Mapping a logical quantum circuit onto physical hardware requires choosing an assignment of logical qubits to physical qubits such that every two-qubit interaction lands on a physically coupled pair. When a perfect assignment exists—that is, when the circuit's interaction graph embeds as a subgraph of the device's coupling graph—finding it is a subgraph isomorphism problem, and both Qiskit and other major

transpilation frameworks attempt exactly this before falling back to heuristic routing with SWAP insertion.

Subgraph isomorphism is NP-complete in general, and practical solvers accordingly impose call and time budgets. What is less appreciated is how sharply the practical difficulty of these instances depends on factors that are invisible to the problem statement: not merely the size of the graphs, but the specific node-traversal order the solver happens to use, and—as we show—which physical qubits happen to host which part of an otherwise identical interaction graph.

This paper characterizes a specific, reproducible failure region in Qiskit’s VF2Layout pass, identifies the ordering strategy responsible, and demonstrates that an extremely simple alternative resolves every instance in the region. We are deliberate about the boundary between what we measured and what we can explain: a substantial part of this paper reports mechanisms we ruled out and regularities we failed to find, because those negative results constrain the problem more sharply than the positive ones alone.

Contributions

1. **Characterization.** A reproducible failure region in Qiskit’s VF2Layout: on saturated instances, median search time of 9236 ms (range 8815–11442 ms) terminating in a “no solution exists” verdict, against 34.1 ms for non-failing instances in the same family—on instances where a valid embedding provably exists (Section 4).
2. **Mechanism, partially.** Qiskit’s VF2Layout uses a custom implementation with hardcoded VF2++ ordering, verified from source, not the public rustworkx mapper. A faithful reimplementation of its Vf2ppSorter shows the ordering defers the symmetric degree-one region of these instances to the end of the traversal (Section 5).
3. **A minimal alternative.** A single rustworkx.vf2_mapping call with id_order=True resolves all 26 instances at a median of 0.0292 ms (Section 6).
4. **Negative results that constrain the problem.** Graph-isomorphic instance pairs diverge reproducibly; no computed graph-level feature explains more than 4.5% of outcome variance; the ordering mechanism alone does not account for instance-level divergence (Section 7).
5. **Methodology.** Every prediction in this work was pre-registered in a timestamped document before the corresponding measurement, including predictions that were subsequently falsified. Several of our own earlier conclusions were corrected in the course of the investigation and the corrections are part of the public record (Section 3).

Background

Layout as subgraph isomorphism

Let $G_I = (V_I, E_I)$ be a circuit's interaction graph, with a vertex per logical qubit and an edge per interacting pair, and let $G_P = (V_P, E_P)$ be the device coupling graph. A *perfect layout* is an injection $\phi : V_I \rightarrow V_P$ with $(\phi(u), \phi(v)) \in E_P$ for every (u,v) in E_I . When such a ϕ exists, no SWAP insertion is required.

We parameterize instances by *spare qubits*: the number of physical qubits not used by the interaction graph. At $\text{spare}=0$ the interaction graph uses every physical qubit, leaving the search no freedom to place a component anywhere unoccupied.

VF2 and VF2++

VF2 [1] explores a search tree of partial mappings, extending one vertex at a time and pruning by local feasibility. Its practical performance depends heavily on the order in which pattern vertices are considered. VF2++ [2] improves on this with a structure-informed ordering: roughly, process vertices in an order that maximizes connectivity to the already-mapped set, breaking ties by degree. On most instance families this ordering is a substantial improvement, which is why it is widely adopted.

Methodology

Software environment

All measurements reported in this paper were taken on AMD64 Family 25 Model 80, with Qiskit 2.5.2 and rustworkx 0.18.1. Section 4's primary measurement (Table 1) used Python 3.10.11; Section 6's ordering-sensitivity measurements (Table 3, and the 0.0292 ms figure) used Python 3.11.9. We report both explicitly rather than a single value, since the two measurements were taken in separate sessions and we did not verify the Python patch version was held constant between them; we have no evidence this affects any reported result, and note it for completeness rather than as a caveat we can currently resolve.

Pre-registration

Every experiment reported here was preceded by a pre-registration document stating the design and the predicted outcomes, including explicit statements of what result would falsify the hypothesis, written and committed before the corresponding measurement was performed. These documents are public and are cited individually below by their addendum number.

This discipline was not cosmetic. Over the course of this investigation, several attractive hypotheses were pre-registered and then falsified by their own designated tests—a component-count divisibility rule, a size-ratio threshold, and a locality-based predictive model among them. Several of our own published intermediate conclusions were subsequently corrected; in each case the correction is recorded alongside the original rather than replacing it. We regard this record as part of the contribution, since the central negative result of Section 7 is only credible to the extent that the positive hypotheses preceding it were genuinely tested rather than abandoned quietly.

Instance families

Our primary family, `dominant_size_sweep`, is parameterized by a single integer D . On $n=64$ qubits it comprises 17 disjoint two-qubit edges (34 qubits) plus one path of D qubits and one path of $30-D$ qubits, for D in $2, \dots, 28$. Every member has zero idle qubits and an interaction-graph maximum matching of exactly 32, equal to that of the 8×8 grid—genuine saturation, verified numerically per instance rather than assumed.

The family has a property we exploit throughout: for any D , the instances at D and $30-D$ are graph-isomorphic, differing only in which physical qubit indices host which path. This gives us matched pairs that any purely structural account must treat identically.

Feasibility

For every instance reported as a failure, we verify independently (via `networkx` maximum matching on the interaction graph against the coupling graph) that a valid embedding exists. Every failure discussed in this paper is a failure to find an embedding that does exist, not a correct report of infeasibility.

The failure region

Primary measurement

Figure 1 shows the qualitative shape of this failure region on an independent, earlier measurement using this project’s original disjoint-pair circuit family (not `dominant_size_sweep`) on the same 6×7 grid: VF2Layout time drops by roughly two orders of magnitude the moment a single spare qubit is introduced, and stays flat from `spare=1` onward. We include it for its visual clarity; the quantitative results in the rest of this section use `dominant_size_sweep` exclusively, and the two families are not compared to each other numerically anywhere in this paper.



VF2Layout time versus spare qubits, 6x7 grid, disjoint-pair circuits, optimization level 3, median of 18 runs per point (3 seeds x 6 repeats), shaded region spans min–max. Independent measurement from Table 1’s dominant_size_sweep results; shown for illustration of the failure region’s shape, not as a quantitative source for this paper’s claims.

Table 1 summarizes VF2Layout at optimization_level=3 on the 8x8 grid across the dominant_size_sweep family, 3 seeds x 2 repeats per instance, replicated in two independent batches.

Qiskit VF2Layout, 8x8 grid, dominant_size_sweep, 18 instances with both batches complete. Every “nonexistent solution” instance admits a valid embedding.

Outcome	Instances	Median time	Range
"nonexistent solution"	13	9236 ms	8815–11442 ms
"solution found"	5	34.1 ms	—
Ratio		271x	

Two features are worth emphasizing. First, the failing instances do not merely take longer—they terminate with a verdict of no solution, on instances where one exists. Downstream, this means the transpiler proceeds to heuristic routing and inserts SWAP gates that were not necessary. Second, the outcome is bimodal: instances land in one regime or the other with no intermediate values, and the classification is stable across independently replicated batches (18/18 instances agreed between batches).

Necessary conditions

Two conditions are jointly necessary for the failure region in our measurements. Introducing a single idle qubit (spare=1) eliminates it. Connecting the interaction graph’s components into a single connected structure also eliminates it. Neither alone

is sufficient: connected interaction graphs at spare=0 do not fail, and disconnected graphs with spare capacity do not fail.

Cross-SDK comparison

We ran an analogous occupancy sweep against Cirq’s own placement search on a 6x7 grid with a 10-second budget per instance. Results are in Table 2.

Cirq placement search, 6x7 grid. Values at spare ≤ 8 are right-censored at the 10-second budget and are lower bounds, not measurements. A valid embedding exists at every row.

Spare qubits	Median elapsed	Timed out
0	≥ 10.03 s	yes
2	≥ 10.03 s	yes
4	≥ 10.03 s	yes
6	≥ 10.03 s	yes
8	≥ 10.02 s	yes
16	2.31 s	no
24	1.40 s	no

We stress the censoring: we cannot report how long Cirq would take, only that it exceeds 10 seconds. The comparison we can make is qualitative but notable in its own right—Cirq’s difficult region extends across spare = 0 through 8, substantially wider than the spare=0-only region we observe in Qiskit. Whatever the underlying cause, it is not unique to one implementation, and its extent differs between implementations.

What Qiskit actually runs

Source-level verification

A natural hypothesis is that Qiskit’s VF2Layout delegates to rustworkx’s public `vf2_mapping`. Reading Qiskit’s current Rust source establishes that it does not. The relevant pass imports only graph data structures from `rustworkx_core`; the search itself is `qiskit_circuit::vf2`, a separate implementation. Both the “average” and “exact” scoring variants construct their search identically with respect to ordering:

```
let vf2 = vf2::Vf2::new(&interactions.graph, &coupling_graph,
                        vf2::Problem::Subgraph)
    .with_scoring(score, score)
    .with_restriction(vf2::Restriction::Decreasing(best_score))
    .with_vf2pp_ordering();
```

`with_vf2pp_ordering()` is called unconditionally in both. We found no code path in this file that runs the search with an alternative ordering, and no configuration option exposing one. This distinction matters for the rest of the paper: the alternative we report in Section 6 is not a setting Qiskit could toggle, but a different implementation.

Simulating the ordering

We reimplemented `Vf2ppSorter`'s ordering rule—select by (connectivity to the already-ordered set, total degree, lowest index), building a BFS tree per root—and ran it on our instances.

The result is unambiguous. In these interaction graphs, path-interior vertices have degree 2 and bare-edge vertices have degree 1. The ordering rule therefore places every path vertex before any bare-edge vertex: for $D=10$ on $n=64$, the first bare-edge vertex appears at position 30 of 64, exactly when the 30-qubit path allocation is exhausted. The symmetric, mutually interchangeable region of the graph is processed last, as a block, after all the structurally distinguishable vertices are placed.

This is consistent with Qiskit's own release notes for version 2.2, which reduced call and trial limits for this pass citing "excessive runtimes for highly symmetric trial circuits being mapped to large coupling maps." The instances in our failure region are exactly that: 17 mutually interchangeable two-qubit components.

Why this is only half a mechanism

The ordering account explains why these instances are hard. It does not explain which of them fail. For the isomorphic pair $D=10$ and $D=20$, the total path allocation is 30 qubits either way, so the ordering produces structurally parallel traversals: both process 30 path vertices, then reach the bare-edge region at position 30. Yet $D=10$ succeeds and $D=20$ fails, reproducibly across independent batches.

Whatever distinguishes them must lie in the backtracking search's behavior once it reaches the tied region, not in the initial ordering. We did not simulate the full backtracking search, and we do not claim to know the answer.

Ordering sensitivity

Result

We ran a single `rustworkx.vf2_mapping` call per instance with `subgraph=True`, `induced=False`, and `id_order=True`—a fixed traversal in natural vertex order, with no structure-informed heuristic at all—on the physical graph in its unmodified numbering.

It finds a valid embedding in **26 of 26** instances, including all 19 where Qiskit reports no solution. Median time: 0.0292 ms; maximum across all instances: 0.108 ms.

Against the failing instances' median of 9236 ms, this is a factor of approximately 3.2×10^5 . We report the comparison with two caveats. It is a comparison between two different implementations, not one implementation under two settings (Section 5); and the Qiskit figure is the time to reach a wrong answer, so the speedup and the correctness improvement are not separable.

Topology generalization

To test whether fixed ordering is merely lucky on square grids, we evaluated it on two structurally different topologies. Because the relevant comparison here is between orderings within a single implementation, both arms use rustworkx: fixed order versus the family of BFS- and degree-based orderings that a prior prototype of ours had used, on the same instances.

Fixed ordering versus structure-informed orderings, disjoint-pair instances at saturation. On brick, the alternatives exhaust nine orderings without success. Heavy-hex instances are saturated relative to the lattice's own maximum matching, since its bipartite imbalance makes qubit-count saturation unattainable.

Topology	n	Fixed order	Time	Other orderings	Time
brick 6x7	42	found	1.25 ms	none found	875 ms
brick 8x8	64	found	1.08 ms	none found	996 ms
heavy-hex d=3	19	found	0.25 ms	—	—
heavy-hex d=5	57	found	0.69 ms	—	—

On brick lattices the contrast is stark: the fixed order succeeds in about a millisecond on instances where nine structure-informed orderings, given a substantially larger budget, find nothing at all.

Interpretation

We do not claim fixed ordering is superior in general; a structure-informed heuristic that is better on typical instances and worse on symmetric ones is an entirely reasonable engineering trade-off, and VF2++ is widely adopted for good reason. What our results establish is narrower and, we think, more useful: on saturated, highly symmetric layout instances—a regime that quantum transpilation specifically cares about, since maximizing device utilization is the point—the sophisticated ordering is not merely slower but qualitatively worse, and the cheapest possible alternative is not merely competitive but dominant by orders of magnitude.

A practical implication follows directly. A transpiler could attempt a budgeted fixed-order pass before its primary search at negligible cost (≤ 0.11 ms in all our measurements) and fall through on failure, gaining the entire failure region at no meaningful expense.

What we could not explain

This section reports negative results. We consider them a contribution rather than a gap: they bound how much of this phenomenon is currently understood, and they close off several natural explanations that a reader would otherwise reasonably propose.

Isomorphic instances diverge

Of the 13 graph-isomorphic pairs (D, 30-D) in our family, 5 diverge: one member succeeds, the other fails, reproducibly. The remaining pairs agree (7 fail on both sides; one is a degenerate construction).

We confirmed isomorphism directly. The pairs differ only in which physical qubit indices host which path. This means **no account based purely on the abstract structure of the interaction graph can be complete**—physical placement, not graph structure alone, determines the outcome.

One regularity survived every test we applied to it: in all 5 diverging pairs, the member with the *smaller* dominant path succeeds. We report this without a proposed mechanism, and note that our record contains several patterns of comparable apparent strength that subsequent tests falsified.

A ratio threshold, falsified

Having observed divergence at size ratios 1.5 and 2.0 and agreement at 2.75 and 5.0, we pre-registered a threshold hypothesis and tested all 13 pairs. The result was non-monotonic: ratios 6.5 and 3.29 diverge while 4.0 and 1.73 agree. The hypothesis is falsified.

No predictive feature

Rather than continue testing candidate variables one at a time—a process that can fail indefinitely without concluding anything—we reframed the question as a measurable one: how much do the features we can compute actually explain?

We computed pairwise distances across all $C(26,2) = 325$ instance pairs under three metrics and correlated each against outcome agreement, with significance assessed by permutation test (5,000 shuffles).

Predictive power of computed graph features. Correction for three tests requires $p < 0.0167$.

Distance metric	r	p	r ²	Survives Bonferroni
Dominant size	-0.151	0.032	2.3%	no
Size ratio	-0.212	0.049	4.5%	no
Max component size	-0.017	0.354	0.0%	no

No metric survives correction, and the largest effect explains under 5% of variance. We state the conclusion precisely: *the graph-level features we computed have little predictive power over this outcome*. This is not a claim that no predictive feature exists. Our instance family has essentially one independent numeric parameter, so no correlational analysis of it can detect structure depending on information that parameter does not carry—including, notably, the physical placement that Section 6.1 shows does matter.

Summary of the boundary

We can state when the failure region occurs (saturation, disconnected interaction graph, sufficient symmetric structure), what ordering Qiskit uses and how it treats these instances, and that a trivial alternative resolves every instance we tested. We cannot predict which specific instances fail, explain why isomorphic instances diverge, or account for the “smaller succeeds” regularity.

Related work

VF2 [1] and VF2++ [2] are the algorithmic basis for the layout passes examined here. Qiskit’s transpiler architecture [3] places VF2Layout before heuristic routing precisely so that perfect layouts, when they exist, are found and SWAP insertion avoided; SABRE [4] is the heuristic fallback when they are not.

Ordering sensitivity in backtracking search is well known in the constraint-satisfaction literature, and symmetric instances are a recognized worst case for subgraph isomorphism generally. Our contribution is not the observation that ordering matters, but the measurement of its magnitude in a specific, deployed, practically important setting—and the finding that the direction of the effect reverses relative to the general case, with the sophisticated heuristic losing to the naive one by five orders of magnitude on the instances that matter most for device utilization.

Conclusion

Layout search in quantum transpilation has a failure region that is sharp, reproducible, and consequential: on saturated instances, Qiskit's VF2Layout spends seconds to reach an incorrect verdict of infeasibility, on instances that a fixed-order search resolves in tens of microseconds. The mechanism is at least partly the node ordering: VF2++'s structure-informed traversal defers the symmetric region of these instances to the end, where the search then struggles.

We have been explicit about the limits of this account. It does not predict which instances fail; isomorphic instances diverge on physical placement alone; no feature we computed explains meaningful outcome variance. We report these as results rather than omissions, because they are what currently bounds understanding of the phenomenon, and because the practical recommendation—attempt a fixed-order pass first, at negligible cost—does not depend on resolving them.

Data and code availability

All raw measurement data, pre-registration documents, analysis scripts, and the complete chronological record including superseded conclusions and their corrections are available at <https://github.com/TN-Holdings-LLC/psf-zero>.

9

L. P. Cordella, P. Foggia, C. Sansone, and M. Vento, *A (sub)graph isomorphism algorithm for matching large graphs*, IEEE Transactions on Pattern Analysis and Machine Intelligence, 26(10):1367–1372, 2004.

A. Juttner and P. Madarasi, *VF2++—An improved subgraph isomorphism algorithm*, Discrete Applied Mathematics, 242:69–81, 2018.

Qiskit contributors, *Qiskit: An Open-source Framework for Quantum Computing*, 2023. <https://qiskit.org>

G. Li, Y. Ding, and Y. Xie, *Tackling the qubit mapping problem for NISQ-era quantum devices*, ASPLOS '19, pp. 1001–1014, 2019.